

```

import torch
import torch.nn as nn
import pandas as pd
import numpy as np
from torch.utils.data import DataLoader, TensorDataset
from sklearn.model_selection import train_test_split

# ----- Load dataset -----
df = pd.read_csv("Music_dataset2.csv")
df.columns = df.columns.str.strip().str.lower()
df["song_id"] = df.index.astype(int)
# Add song popularity feature
df["popularity_score"] = np.random.uniform(0.3, 1.0, len(df))
num_songs = len(df)
seq_len = 10

# ----- Genre & Vibe Mapping -----
# Use 'genres' column from your Spotify dataset
if "genres" in df.columns:
    df["genre"] = df["genres"].fillna("Unknown")
elif "genre" in df.columns:
    df["genre"] = df["genre"].fillna("Unknown")
else:
    df["genre"] = "Unknown"

if "vibe" in df.columns:
    df["vibe"] = df["vibe"].fillna("Neutral")
else:
    df["vibe"] = "Neutral"

genre_map = {g: i for i, g in enumerate(df["genre"].unique())}
vibe_map = {v: i for i, v in enumerate(df["vibe"].unique())}

df["genre_id"] = df["genre"].map(genre_map)
df["vibe_id"] = df["vibe"].map(vibe_map)

num_genres = len(genre_map)
num_vibes = len(vibe_map)

# ----- REAL INTERACTIONS FROM XLSX FILES -----
import os

xlsx_files = [
    "Keziah_trainingsheet.xlsx",
    "Kevin_trainingsheet.xlsx",
    "Sherly_trainingsheet.xlsx",
    "Vinod_trainingsheet.xlsx",
    "Aadesh_trainingsheet.xlsx",

```

```

    "AdithyaKrishna_trainingsheet.xlsx",
    "Anil_trainingsheet.xlsx",
    "Dnyana_Training sheet.xlsx"
]

#Columns of ONE Excel file:
# print(pd.read_excel("Keziah_trainingsheet.xlsx").columns.tolist())

all_users_data = []
user_id = 0

# Detect song column in music dataset
# print("Music CSV columns:", df.columns.tolist())

# Detect song column in music dataset (UPDATED FOR YOUR CSV)
# if "track name" in df.columns:
#     music_song_col = "Track Name"
# elif "song" in df.columns:
#     music_song_col = "song"
# elif "song_name" in df.columns:
#     music_song_col = "song_name"
# elif "title" in df.columns:
#     music_song_col = "title"
# else:
#     raise Exception("No song column found in Music_dataset1.csv")

if "track name" not in df.columns:
    raise Exception("No 'Track Name' column found in Music_dataset1.csv")

music_song_col = "track name"

for file in xlsx_files:
    if not os.path.exists(file):
        print(f"{file} not found, skipping...")
        continue

    # Get all sheet names
    xl = pd.ExcelFile(file)
    sheet_names = xl.sheet_names # reads ALL sheets
    print(f"{file}: found {len(sheet_names)} sheets")

    for sheet in sheet_names:
        user_df = pd.read_excel(file, sheet_name=sheet, header=None)
        raw = user_df.astype(str)

        # ----- Extract context features from sheet -----
        hrv = 0.5
        stress = 0.5

```

```
tipi = 0.5
dass21 = 0.5
whoqol = 0.5
```

```
for i in range(len(raw)):
    for j in range(len(raw.columns)):
        cell = str(raw.iloc[i, j]).lower()
```

```
    if "hrv" in cell or "heart rate variability" in cell:
        try:
            hrv = float(raw.iloc[i, j+1]) / 100
        except:
            hrv = 0.5
```

```
    if "stress" in cell:
        try:
            stress = float(raw.iloc[i, j+1]) / 100
        except:
            stress = 0.5
```

```
    if "tipi" in cell:
        try:
            tipi = float(raw.iloc[i, j+1]) / 10
        except:
            tipi = 0.5
```

```
    if "dass" in cell:
        try:
            dass21 = float(raw.iloc[i, j+1]) / 42
        except:
            dass21 = 0.5
```

```
    if "whoqol" in cell or "quality of life" in cell:
        try:
            whoqol = float(raw.iloc[i, j+1]) / 100
        except:
            whoqol = 0.5
```

```
# ----- Generate interactions (same as before) -----
```

```
if stress > 0.7:
    candidate_songs = df[df["energy"] < 0.4]
elif stress < 0.3:
    candidate_songs = df[df["energy"] > 0.6]
else:
    candidate_songs = df[(df["energy"] >= 0.4) & (df["energy"] <= 0.6)]
```

```
if len(candidate_songs) < 60:
    candidate_songs = df
```

```

sampled_songs = candidate_songs.sample(60, replace=True)
mood = int(np.clip(round((whoqol + (1 - stress) + tipi) / 3 * 4), 0, 4))

matched_rows = []
for _, row in sampled_songs.iterrows():
    song_energy = row["energy"]
    song_valence = row["valence"]
    rating = 3
    if stress > 0.7 and song_energy < 0.4:                rating += 2
    elif stress < 0.3 and song_energy > 0.6:              rating += 2
    if song_valence > 0.6:                                rating += 1
    if tipi > 0.6 and song_energy > 0.6:                  rating += 1
    if dass21 > 0.5 and song_energy < 0.4 and song_valence < 0.4: rating += 1
    if whoqol < 0.4 and song_valence > 0.6:              rating += 1
    rating = int(np.clip(rating, 1, 5))

```

```

matched_rows.append({
    "song_id": row["song_id"],
    "genre_id": row["genre_id"],
    "vibe_id": row["vibe_id"],
    "mood": mood,
    "stress": stress,
    "hrv": hrv,
    "tipi": tipi,
    "dass21": dass21,
    "whoqol": whoqol,
    "popularity": row["popularity_score"],
    "rating": rating
})

```

```

merged = pd.DataFrame(matched_rows)
merged["user_id"] = user_id
merged["timestamp"] = np.arange(len(merged))
all_users_data.append(merged)
user_id += 1

```

```

# =====
# GOOGLE FORM INTEGRATION
# Place this block RIGHT AFTER your xlsx loop ends,
# BEFORE the final pd.concat() line
# =====

```

GOOGLE_FORM_FILE = "Dataset collection (Responses) (1).xlsx" # ← your actual filename

```

def parse_google_form_xlsx(filepath):
    """

```

Parses a structured Google Form XLSX export with raw question scores.

Expected columns (Google Form exports use full question text as headers):

- Name / Age (ignored)
- HRV → any column containing 'hrv'
- Stress Level → any column containing 'stress'
- TIPI Q1–Q10 → columns containing 'tipi' (scale 1–7 each)
- DASS-21 Q1–Q21 → columns containing 'dass' (scale 0–3 each)
- WHOQOL Q1–Q26 → columns containing 'whoqol' or 'quality of life' (scale 1–5 each)

"""

```
gf = pd.read_excel(filepath)
```

```
gf.columns = gf.columns.str.strip().str.lower()
```

```
# print(f"\n📋 Google Form columns detected:\n{gf.columns.tolist()}\n")
```

```
users = []
```

```
for _, row in gf.iterrows():
```

```
    # --- HRV ---
```

```
    hrv = 0.5
```

```
    hrv_cols = [c for c in gf.columns if "hrv" in c]
```

```
    if hrv_cols:
```

```
        try:
```

```
            val = float(row[hrv_cols[0]])
```

```
            # Auto-detect scale: if value > 1, assume it's 0–100 or raw ms
```

```
            hrv = val / 100 if val > 1 else val
```

```
        except:
```

```
            hrv = 0.5
```

```
    # --- Stress Level ---
```

```
    stress = 0.5
```

```
    stress_cols = [c for c in gf.columns if "stress" in c]
```

```
    if stress_cols:
```

```
        try:
```

```
            val = float(row[stress_cols[0]])
```

```
            # Auto-detect scale: 0–10 or 0–100
```

```
            if val > 10:
```

```
                stress = val / 100
```

```
            else:
```

```
                stress = val / 10
```

```
        except:
```

```
            stress = 0.5
```

```
    # --- TIPI: 10 items, each scored 1–7 → normalize by (10 × 7 = 70) ---
```

```
    tipi = 0.5
```

```
    tipi_cols = [c for c in gf.columns if "tipi" in c]
```

```
    if tipi_cols:
```

```

try:
    scores = pd.to_numeric(row[tipi_cols], errors="coerce").fillna(4)
    tipi = float(scores.sum()) / (len(tipi_cols) * 7)
except:
    tipi = 0.5
else:
    print("⚠ No TIPI columns found — check column names contain 'tipi'")

# --- DASS-21: 21 items, each scored 0–3 → normalize by (21 × 3 = 63) ---
dass21 = 0.5
dass_cols = [c for c in gf.columns if "dass" in c]
if dass_cols:
    try:
        scores = pd.to_numeric(row[dass_cols], errors="coerce").fillna(1)
        dass21 = float(scores.sum()) / (len(dass_cols) * 3)
    except:
        dass21 = 0.5
else:
    print("⚠ No DASS-21 columns found — check column names contain 'dass'")

# --- WHOQOL-BREF: 26 items, each scored 1–5 → normalize by (26 × 5 = 130) ---
whoqol = 0.5
whoqol_cols = [c for c in gf.columns if "whoqol" in c or "quality of life" in c]
if whoqol_cols:
    try:
        scores = pd.to_numeric(row[whoqol_cols], errors="coerce").fillna(3)
        whoqol = float(scores.sum()) / (len(whoqol_cols) * 5)
    except:
        whoqol = 0.5
else:
    print("⚠ No WHOQOL columns found — check column names contain 'whoqol'")

users.append({
    "hrv": np.clip(hrv, 0.0, 1.0),
    "stress": np.clip(stress, 0.0, 1.0),
    "tipi": np.clip(tipi, 0.0, 1.0),
    "dass21": np.clip(dass21, 0.0, 1.0),
    "whoqol": np.clip(whoqol, 0.0, 1.0),
})

return users

# ---- Load Google Form responses ----
if os.path.exists(GOOGLE_FORM_FILE):
    google_form_users = parse_google_form_xlsx(GOOGLE_FORM_FILE)
    print(f"✅ Loaded {len(google_form_users)} Google Form responses")

```

```

# ---- Generate interactions (same rating logic as xlsx block) ----
for gf_user in google_form_users:
    hrv = gf_user["hrv"]
    stress = gf_user["stress"]
    tipi = gf_user["tipi"]
    dass21 = gf_user["dass21"]
    whoqol = gf_user["whoqol"]

    # Candidate song selection based on stress
    if stress > 0.7:
        candidate_songs = df[df["energy"] < 0.4]
    elif stress < 0.3:
        candidate_songs = df[df["energy"] > 0.6]
    else:
        candidate_songs = df[(df["energy"] >= 0.4) & (df["energy"] <= 0.6)]

    if len(candidate_songs) < 60:
        candidate_songs = df

    sampled_songs = candidate_songs.sample(60, replace=True)
    mood = int(np.clip(round((whoqol + (1 - stress) + tipi) / 3 * 4), 0, 4))

    matched_rows = []
    for _, row in sampled_songs.iterrows():
        song_energy = row["energy"]
        song_valence = row["valence"]
        rating = 3
        if stress > 0.7 and song_energy < 0.4: rating += 2
        elif stress < 0.3 and song_energy > 0.6: rating += 2
        if song_valence > 0.6: rating += 1
        if tipi > 0.6 and song_energy > 0.6: rating += 1
        if dass21 > 0.5 and song_energy < 0.4 and song_valence < 0.4: rating += 1
        if whoqol < 0.4 and song_valence > 0.6: rating += 1
        rating = int(np.clip(rating, 1, 5))

    matched_rows.append({
        "song_id": row["song_id"],
        "genre_id": row["genre_id"],
        "vibe_id": row["vibe_id"],
        "mood": mood,
        "stress": stress,
        "hrv": hrv,
        "tipi": tipi,
        "dass21": dass21,
        "whoqol": whoqol,
        "popularity": row["popularity_score"],
        "rating": rating
    })

```

```

merged = pd.DataFrame(matched_rows)
merged["user_id"] = user_id # continues from xlsx user_id counter
merged["timestamp"] = np.arange(len(merged))
all_users_data.append(merged)
user_id += 1

print(f"✅ Total users after Google Form: {user_id} "
      f"(8 xlsx + {len(google_form_users)} form responses)")

else:
    print(f"⚠️ {GOOGLE_FORM_FILE} not found — skipping Google Form data")

# ---- pd.concat() follows here (already in your code) ----
# inter = pd.concat(all_users_data, ignore_index=True)

# Final interaction dataset used by RNN + NCF
if len(all_users_data) == 0:
    print(f"⚠️ No song logs found in Excel. Creating physiology-based interactions...")

interactions = []
users_data = []

# ----- STEP 1: Extract HRV & Stress from Excel -----
for file in xlsx_files:
    if not os.path.exists(file):
        continue

    raw = pd.read_excel(file, header=None)

    hrv = 0.5
    stress = 0.5

    for i in range(len(raw)):
        for j in range(len(raw.columns)):
            cell = str(raw.iloc[i, j]).lower()

            if "hrv" in cell or "heart rate" in cell or "hr:" in cell:
                try:
                    hrv = float(raw.iloc[i, j+1]) / 100
                except:
                    hrv = 0.5

            if "stress" in cell:
                try:
                    stress = float(raw.iloc[i, j+1]) / 100
                except:
                    stress = 0.5

```



```
users_data.append({"hrv": hrv, "stress": stress})
```

```
# ----- STEP 2: Create REAL interactions using physiology -----
```

```
for user_id, user in enumerate(users_data):
```

```
    hrv = user["hrv"]
```

```
    stress = user["stress"]
```

```
# 50 interactions per user (good for RNN + NCF training)
```

```
sampled_songs = df.sample(50, random_state=42)
```

```
for _, row in sampled_songs.iterrows():
```

```
    # Smart physiological rating logic
```

```
    if stress < 0.3:
```

```
        rating = 5
```

```
    elif stress < 0.6:
```

```
        rating = 4
```

```
    else:
```

```
        rating = 3
```

```
mood = int(np.clip(5 - stress*4, 1, 5))
```

```
interactions.append([
```

```
    user_id,
```

```
    row["song_id"],
```

```
    row["genre_id"],
```

```
    row["vibe_id"],
```

```
    mood,
```

```
    stress,
```

```
    hrv,
```

```
    rating
```

```
])
```

```
inter = pd.DataFrame(
```

```
    interactions,
```

```
    columns=[
```

```
        "user_id", "song_id", "genre_id", "vibe_id",
```

```
        "mood", "stress", "hrv", "rating"
```

```
    ]
```

```
)
```

```
# Extract HRV and Stress from sheet text
```

```
for i in range(len(row)):
```

```
    for j in range(len(row.columns)):
```

```
        cell = str(row.iloc[i, j]).lower()
```

```
        if "hrv" in cell or "heart rate" in cell:
```

```
            try:
```

```
                hrv = float(row.iloc[i, j+1]) / 100
```

```

        except:
            hrv = 0.5
    if "stress" in cell:
        try:
            stress = float(raw.iloc[i, j+1]) / 100
        except:
            stress = 0.5

    mood = int(np.clip(3 - stress*2, 1, 5))

    # Generate synthetic listening history (VERY IMPORTANT for RNN)
    history = np.random.choice(df["song_id"], size=80, replace=False)

    for song_id in history:
        row = df.loc[song_id]
        interactions.append([
            user_id,
            song_id,
            row["genre_id"],
            row["vibe_id"],
            mood,
            stress,
            hrv,
            np.random.randint(2, 6)
        ])

    inter = pd.DataFrame(
        interactions,
        columns=[
            "user_id", "song_id", "genre_id", "vibe_id",
            "mood", "stress", "hrv", "rating"
        ]
    )
else:
    inter = pd.concat(all_users_data, ignore_index=True)

print("Final training interactions:", inter.shape)

# Normalize mood to 0-1
# inter["mood"] = inter["mood"] / 5.0
mood = int(np.clip(5 - stress*4, 1, 5))

if inter.empty:
    print("No valid song matches found in Excel. Creating fallback synthetic interactions...")

n_samples = 3000 # Good for RNN + NCF training

inter = pd.DataFrame({

```

```

        "user_id": np.random.randint(0, 10, n_samples),
        "song_id": np.random.randint(0, num_songs, n_samples),
        "genre_id": np.random.randint(0, num_genres, n_samples),
        "vibe_id": np.random.randint(0, num_vibes, n_samples),
        "mood": np.random.randint(1, 5, n_samples),
        "stress": np.random.uniform(0.2, 0.8, n_samples),
        "hrv": np.random.uniform(0.4, 1.0, n_samples),
        "rating": np.random.randint(2, 6, n_samples)
    })

    print("Fallback interactions created:", inter.shape)

# ===== ContextRNN =====
class ContextRNN(nn.Module):
    def __init__(self, num_songs, num_genres, num_vibes, embed_dim=128,
hidden_dim=64):
        super().__init__()
        self.song_emb = nn.Embedding(num_songs, embed_dim)
        self.genre_emb = nn.Embedding(num_genres, 8)
        self.vibe_emb = nn.Embedding(num_vibes, 8)
        self.context_fc = nn.Linear(6, 16) # mood, stress, hrv
        self.lstm = nn.LSTM(160, 64, batch_first=True)
        self.dropout = nn.Dropout(0.5)
        self.attention = nn.Linear(64, 1)
        self.fc = nn.Linear(64, num_songs)

    def forward(self, seq, genre, vibe, ctx):
        s = self.song_emb(seq)
        g = self.genre_emb(genre).unsqueeze(1).repeat(1, seq.size(1), 1)
        v = self.vibe_emb(vibe).unsqueeze(1).repeat(1, seq.size(1), 1)
        c = self.context_fc(ctx).unsqueeze(1).repeat(1, seq.size(1), 1)
        x = torch.cat([s, g, v, c], dim=2)
        out, _ = self.lstm(x)
        out = self.dropout(out)
        attn_weights = torch.softmax(self.attention(out), dim=1) #Attention weights
        context = torch.sum(attn_weights * out, dim=1) # Weighted context vector
        return self.fc(context)

if len(inter) < seq_len + 1:
    raise ValueError(
        "Not enough interaction data for RNN. Need at least sequence length + 1 samples."
    )

# ---- Prepare RNN data (FIXED: per user sequences) ----
seqs, genres, vibes, ctxs, nexts = [], [], [], [], []

for uid in inter.user_id.unique():

```

```

user_data = inter[inter.user_id == uid].sort_values("timestamp").reset_index(drop=True)

# Sort if you have timestamp column
# user_data = user_data.sort_values("timestamp")

if len(user_data) <= seq_len:
    continue

for i in range(len(user_data) - seq_len):

    seqs.append(
        user_data.song_id.iloc[i:i+seq_len].tolist()
    )

    genres.append(
        user_data.genre_id.iloc[i+seq_len]
    )

    vibes.append(
        user_data.vibe_id.iloc[i+seq_len]
    )

    ctxs.append([
        user_data.mood.iloc[i+seq_len],
        user_data.stress.iloc[i+seq_len],
        user_data.hrv.iloc[i+seq_len],
        user_data.tipi.iloc[i+seq_len],
        user_data.dass21.iloc[i+seq_len],
        user_data.whoqol.iloc[i+seq_len]
    ])

    nexts.append(
        user_data.song_id.iloc[i+seq_len]
    )

dataset = TensorDataset(
    torch.tensor(seqs, dtype=torch.long),
    torch.tensor(genres),
    torch.tensor(vibes),
    torch.tensor(ctxs, dtype=torch.float32),
    torch.tensor(nexts, dtype=torch.long)
)

from sklearn.model_selection import train_test_split

seqs = np.array(seqs)
genres = np.array(genres)
vibes = np.array(vibes)

```

```

ctxs = np.array(ctxs)
nexts = np.array(nexts)

# Split 80% train, 20% validation
s_train, s_val, g_train, g_val, v_train, v_val, c_train, c_val, y_train, y_val = train_test_split(
    seqs, genres, vibes, ctxs, nexts, test_size=0.2, random_state=42
)

train_dataset = TensorDataset(
    torch.tensor(s_train, dtype=torch.long),
    torch.tensor(g_train, dtype=torch.long),
    torch.tensor(v_train, dtype=torch.long),
    torch.tensor(c_train, dtype=torch.float32),
    torch.tensor(y_train, dtype=torch.long)
)

val_dataset = TensorDataset(
    torch.tensor(s_val, dtype=torch.long),
    torch.tensor(g_val, dtype=torch.long),
    torch.tensor(v_val, dtype=torch.long),
    torch.tensor(c_val, dtype=torch.float32),
    torch.tensor(y_val, dtype=torch.long)
)

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False)

num_songs = len(df)
num_genres = len(genre_map)
num_vibes = len(vibe_map)

rnn = ContextRNN(num_songs, num_genres, num_vibes)
opt = torch.optim.Adam(rnn.parameters(), lr=0.001)
loss_fn = nn.CrossEntropyLoss()

opt = torch.optim.Adam(rnn.parameters(), lr=0.001, weight_decay=1e-4)

best_val_loss = float('inf')
patience = 4
patience_counter = 0

for epoch in range(30):
    # --- Training ---
    rnn.train()
    total_loss = 0
    for s, g, v, c, y in train_loader:
        opt.zero_grad()
        out = rnn(s, g, v, c)

```

```

    loss = loss_fn(out, y)
    loss.backward()
    opt.step()
    total_loss += loss.item()
    avg_train_loss = total_loss / len(train_loader)

# --- Validation ---
rnn.eval()
val_loss = 0
with torch.no_grad():
    for s, g, v, c, y in val_loader:
        out = rnn(s, g, v, c)
        loss = loss_fn(out, y)
        val_loss += loss.item()
    avg_val_loss = val_loss / len(val_loader)

print(f"RNN Epoch {epoch+1} | Train Loss: {avg_train_loss:.4f} | Val Loss: {avg_val_loss:.4f}")

# Early stopping — 4 spaces, same level as print above
if avg_val_loss < best_val_loss - 0.001:
    best_val_loss = avg_val_loss
    patience_counter = 0
    torch.save(rnn.state_dict(), "rnn_model_trained3.pth") # saves best epoch only
else:
    patience_counter += 1
    if patience_counter >= patience:
        print(f"Early stopping at epoch {epoch+1} — best val loss: {best_val_loss:.4f}")
        break

# ===== ContextNCF =====
class ContextNCF(nn.Module):
    def __init__(self, num_users, num_songs, num_genres, num_vibes):
        super().__init__()
        self.user_emb = nn.Embedding(num_users, 16) # user embedding
        self.song_emb = nn.Embedding(num_songs, 32)
        self.genre_emb = nn.Embedding(num_genres, 8)
        self.vibe_emb = nn.Embedding(num_vibes, 8)
        self.mood_emb = nn.Embedding(6, 4)

        total_input = 16 + 32 + 8 + 8 + 4 + 1 + 1 + 1 + 1 + 1 + 1 # 16 from user

        self.fc1 = nn.Linear(total_input, 64)
        self.fc2 = nn.Linear(64, 32)
        self.out = nn.Linear(32, 1)

    def forward(self, u, s, g, v, m, st, h, t, d, w, pop):
        x = torch.cat([

```

```

        self.user_emb(u),
        self.song_emb(s),
        self.genre_emb(g),
        self.vibe_emb(v),
        self.mood_emb(m),
        st,
        h,
        t,
        d,
        w,
        pop
    ], dim=1)

    x = torch.relu(self.fc1(x))
    x = torch.relu(self.fc2(x))
    return self.out(x)

# ----- Create NCF Dataset -----
num_users = inter.user_id.nunique()

ncf_dataset = TensorDataset(
    torch.tensor(inter.user_id.values, dtype=torch.long), # new
    torch.tensor(inter.song_id.values, dtype=torch.long),
    torch.tensor(inter.genre_id.values, dtype=torch.long),
    torch.tensor(inter.vibe_id.values, dtype=torch.long),
    torch.tensor(inter.mood.values, dtype=torch.long),
    torch.tensor(inter.stress.values, dtype=torch.float32).unsqueeze(1),
    torch.tensor(inter.hrv.values, dtype=torch.float32).unsqueeze(1),
    torch.tensor(inter.tipi.values, dtype=torch.float32).unsqueeze(1),
    torch.tensor(inter.dass21.values, dtype=torch.float32).unsqueeze(1),
    torch.tensor(inter.whoqi.values, dtype=torch.float32).unsqueeze(1),
    torch.tensor(inter.popularity.values, dtype=torch.float32).unsqueeze(1),
    torch.tensor(inter.rating.values, dtype=torch.float32).unsqueeze(1)
)

# Split indices
train_idx, val_idx = train_test_split(np.arange(len(inter)), test_size=0.2, random_state=42)

train_data = inter.iloc[train_idx]
val_data = inter.iloc[val_idx]

# TensorDataset for training
ncf_train_dataset = TensorDataset(
    torch.tensor(train_data.user_id.values, dtype=torch.long),
    torch.tensor(train_data.song_id.values, dtype=torch.long),
    torch.tensor(train_data.genre_id.values, dtype=torch.long),
    torch.tensor(train_data.vibe_id.values, dtype=torch.long),
    torch.tensor(train_data.mood.values, dtype=torch.long),

```

```

torch.tensor(train_data.stress.values, dtype=torch.float32).unsqueeze(1),
torch.tensor(train_data.hrv.values, dtype=torch.float32).unsqueeze(1),
torch.tensor(train_data.tipi.values, dtype=torch.float32).unsqueeze(1),
torch.tensor(train_data.dass21.values, dtype=torch.float32).unsqueeze(1),
torch.tensor(train_data.whoqol.values, dtype=torch.float32).unsqueeze(1),
torch.tensor(train_data.popularity.values, dtype=torch.float32).unsqueeze(1),
torch.tensor(train_data.rating.values, dtype=torch.float32).unsqueeze(1)
)

```

```

ncf_val_dataset = TensorDataset(
    torch.tensor(val_data.user_id.values, dtype=torch.long),
    torch.tensor(val_data.song_id.values, dtype=torch.long),
    torch.tensor(val_data.genre_id.values, dtype=torch.long),
    torch.tensor(val_data.vibe_id.values, dtype=torch.long),
    torch.tensor(val_data.mood.values, dtype=torch.long),
    torch.tensor(val_data.stress.values, dtype=torch.float32).unsqueeze(1),
    torch.tensor(val_data.hrv.values, dtype=torch.float32).unsqueeze(1),
    torch.tensor(val_data.tipi.values, dtype=torch.float32).unsqueeze(1),
    torch.tensor(val_data.dass21.values, dtype=torch.float32).unsqueeze(1),
    torch.tensor(val_data.whoqol.values, dtype=torch.float32).unsqueeze(1),
    torch.tensor(val_data.popularity.values, dtype=torch.float32).unsqueeze(1),
    torch.tensor(val_data.rating.values, dtype=torch.float32).unsqueeze(1)
)

```

```

train_loader = DataLoader(ncf_train_dataset, batch_size=32, shuffle=True)
val_loader = DataLoader(ncf_val_dataset, batch_size=32, shuffle=False)

```

```

ncf = ContextNCF(num_users, num_songs, num_genres, num_vibes)
opt = torch.optim.Adam(ncf.parameters(), lr=0.0005)
loss_fn = nn.MSELoss()

```

```

for epoch in range(15):
    ncf.train()
    total_train_loss = 0
    for u, s, g, v, m, st, h, t, d, w, pop, y in train_loader:
        opt.zero_grad()
        pred = ncf(u, s, g, v, m, st, h, t, d, w, pop)
        loss = loss_fn(pred, y)
        loss.backward()
        opt.step()
        total_train_loss += loss.item()

```

```

avg_train_loss = total_train_loss / len(train_loader)

```

```

# Validation
ncf.eval()
total_val_loss = 0
with torch.no_grad():

```



```

    for u, s, g, v, m, st, h, t, d, w, pop, y in val_loader:
        pred = ncf(u, s, g, v, m, st, h, t, d, w, pop)
        loss = loss_fn(pred, y)
        total_val_loss += loss.item()

    avg_val_loss = total_val_loss / len(val_loader)
    print(f"NCF Epoch {epoch+1} | Train Loss: {avg_train_loss:.4f} | Val Loss: {avg_val_loss:.4f}")

torch.save(ncf.state_dict(), "ncf_model_trained3.pth")

# ----- Save Training Metadata -----
metadata = {
    "num_users": num_users,
    "num_songs": num_songs,
    "num_genres": num_genres,
    "num_vibes": num_vibes,
    "genre_mapping": genre_map,
    "vibe_mapping": vibe_map
}

torch.save(metadata, "metadata2.pth")

```